

Manual de Ingeniería y Especificación Declarativa de Subagentes para Antigravity 2.0: Proyecto pc-custom-lab

1. Arquitectura de Aislamiento y Gestión de Contexto de Próxima Generación

La evolución del ecosistema Google Antigravity 2.0 marca el fin de la era de la programación imperativa de agentes, consolidando la orquestación declarativa como el estándar para entornos de producción masiva. En el proyecto pc-custom-lab, que gestiona simultáneamente la lógica y despliegue de 6 tiendas comerciales, la arquitectura debe priorizar la protección de la ventana de contexto. El paso de scripts rígidos a archivos de configuración YAML permite establecer fronteras normativas auditables, donde el orquestador no se ve saturado por el razonamiento intermedio, sino que mantiene una visibilidad ejecutiva delegando la carga operativa a subunidades especializadas. En Windows 11, esta transición es crítica para garantizar que la ejecución local sobre terminales de alto rendimiento no degrade la precisión del modelo debido a la acumulación de trazas históricas.

1.1 Sidechain Transcripts y Persistencia Dual

Antigravity resuelve la saturación de memoria mediante Sidechain Transcripts (Transcripciones en Cadena Lateral). A diferencia de los sistemas tradicionales que inyectan todo el historial en el hilo principal, Antigravity desacopla la actividad del subagente en dos artefactos de persistencia:

.jsonl (Eventos Secuenciales): Un registro persistente de mensajes, llamadas a herramientas y snapshots del sistema.

.meta.json (Metadatos de Estado): Almacena la configuración específica y el estado actual de la tarea.

El mecanismo de "summary-only return" es el componente vital de este diseño: una vez concluida la labor, el sistema extrae un resumen sintético que se reinyecta en el orquestador raíz. Esto previene la "degradación informativa", asegurando que el presupuesto de tokens del orquestador se reserve para la toma de decisiones de alto nivel y no para procesar logs de compilación de las 6 tiendas.

1.2 Evaluación de Herramientas: SkillTool vs. AgentTool

La distinción entre herramientas directas e instanciación de agentes es puramente estratégica:

Característica

SkillTool

AgentTool

Latencia

Mínima (Ejecución inmediata)

Mayor (Requiere instanciación)

Consumo de Tokens

Expansivo (Suma al hilo principal)

Optimizado (Contexto aislado)

Aislamiento de Memoria

Bajo (Comparte ventana principal)

Total (Sidechain independiente)

Factor de Sobrecosto

1x

Hasta 7x (en razonamiento intermedio)

Caso de Uso Óptimo

Lectura de archivos y consultas rápidas

Tareas complejas, refactorización masiva

1.3 Pipeline de Moldeadores de Contexto (Pre-model Shapers)

Antes de invocar al modelo Gemini 3.7 Flash, el motor de Antigravity ejecuta un pipeline secuencial de cinco capas para optimizar la coherencia:

Budget Reduction: Evalúa límites físicos de memoria y descarta elementos obsoletos.

Snip: Recorta respuestas antiguas de herramientas para priorizar la actualidad operativa.

Microcompact: Elimina redundancias sintácticas y espacios en blanco en las trazas.

Context Collapse: Realiza una proyección sintética no destructiva de la memoria en tiempo de lectura.

Auto-Compact: Aplica un resumen integral guiado por un modelo auxiliar al superar umbrales críticos.

Capa "So What?": La ejecución secuencial de estos moldeadores es la única garantía de mantener la coherencia lógica en refactorizaciones de larga duración. Sin este pipeline, el modelo perdería el "foco" sobre las reglas de negocio de los 6 pares de tiendas, resultando en alucinaciones de código o errores de sintaxis catastróficos.

2. Protocolo de Seguridad Estricta y Matriz de Permisos

En entornos locales sobre Windows 11, la seguridad de Antigravity 2.0 se rige por la filosofía "Deny-First". Bajo este estándar, cualquier acción no declarada explícitamente se considera una amenaza potencial y se bloquea por defecto.

2.1 Mitigación de Vulnerabilidades Críticas (CVE-2026-12958)

El sistema neutraliza el vector de ataque GhostApproval. Esta vulnerabilidad permitía engañar al usuario mediante enlaces simbólicos (symlinks) que apuntaban a archivos sensibles (como claves .ssh o configuraciones de entorno). Antigravity elimina este riesgo mediante la resolución canónica de rutas: el sistema resuelve la ruta física real de cualquier archivo antes de presentar el diálogo de aprobación, garantizando que el usuario autorice exactamente lo que el agente ve.

2.2 Configuración de Aislamiento de Ejecución

Para pc-custom-lab, implementamos `isolation_mode: worktree`. Este modo utiliza las capacidades nativas de Git para crear árboles de trabajo secundarios. Esto permite que los subagentes ejecuten comandos y modifiquen archivos en directorios aislados, protegiendo el repositorio principal de estados inconsistentes durante las pruebas de integración.

2.3 Ciclo de Vida de Hooks y Autorización

La seguridad se refuerza mediante `.agents/hooks.json`, utilizando hooks `pre_tool_use` para validación determinista. Adicionalmente, el sistema emplea el `yoloClassifier`, un clasificador basado en ML de dos etapas (filtro rápido y cadena de pensamiento) que evalúa el riesgo de cada acción antes de solicitar permiso al usuario.

Capa "So What?": La superioridad de la política `disallowedTools` sobre las listas de permisos tradicionales reside en su capacidad de prevención de escalada de privilegios. Al prohibir herramientas como `execute_bash` en subagentes específicos, creamos compartimentos estancos donde un fallo en un componente de frontend no puede comprometer la integridad del sistema operativo anfitrión.

3. Especificación Declarativa de Agentes (Ecosistema pc-custom-lab)

La arquitectura basada en el esquema `AgentConfig.json` de ADK 1.12.0 permite definir identidades sin escribir código Python, garantizando portabilidad total en Windows 11.

3.1 Orquestador Raíz: Director General (.agents/agents.yaml)

```
# yaml-language-server: $schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google/adk/agents/config_schemas/AgentConfig.json
```

```
name: director_general
```

```
agent_class: LlmAgent
```

```
model: gemini-3.7-flash
```

```
effort: high
```

```
description: Orquestador raíz de pc-custom-lab. Coordina 6 tiendas comerciales.
```

```
instruction: |
```

```
Usted es el Director General. Su función es delegar tareas a especialistas  
y mantener la integridad del sistema. Nunca realice tareas de edición masiva;  
siempre utilice AgentTool para delegar en subagentes especializados.
```

```
sub_agents:
```

- config_path: ./subagents/arquitecto_web.yaml
- config_path: ./subagents/desarrollador_frontend.yaml
- config_path: ./subagents/gestor_datos.yaml
- config_path: ./subagents/compilador_tailwind.yaml
- config_path: ./subagents/optimizador_mobile.yaml
- config_path: ./subagents/seguridad_backups.yaml
- config_path: ./subagents/auditor_qa.yaml
- config_path: ./subagents/ingeniero_python.yaml
- config_path: ./subagents/gestor_git.yaml

```
permissionMode: default
```

```
isolation_mode: in-process
```

3.2 Especialistas de Estructura y Frontend

Arquitecto Web:

name: arquitecto_web

model: gemini-3.7-flash

instruction: Diseña y valida la jerarquía de componentes React para las 6 tiendas.

Desarrollador Frontend:

name: desarrollador_frontend

model: gemini-3.7-flash

instruction: |

Implementa lógica de cotización para 6 pares de tiendas.

Gestiona sliders de presupuesto con debounce para optimizar rendimiento.

3.3 Gestión de Datos y Estilos (Windows Native)

Gestor de Datos:

name: gestor_datos

model: gemini-3.7-flash

instruction: |

Protege la persistencia del Drawer del carrito y localStorage.

Asegura que los datos de sesión sean consistentes entre las 6 instancias.

Compilador Tailwind:

name: compilador_tailwind

tools:

- name: execute_bash

instruction: |

Ejecuta npx.cmd tailwindcss para regenerar estilos en Windows 11.

Debe usar explícitamente npx.cmd para evitar fallos de resolución en CMD/PowerShell.

3.4 Optimización, Seguridad y QA

Optimizador Mobile UX:

name: optimizador_mobile_ux

instruction: Valida que los 6 frontends mantengan adaptabilidad móvil fluida.

Seguridad y Backups:

name: seguridad_backups

instruction: |

Obligatoriedad de crear archivos .bak mediante hooks antes de cada escritura.

Valida la integridad de los backups tras cada operación de refactorización.

Auditor QA:

name: auditor_qa

instruction: |

Ejecuta suites de prueba e inspecciona estrictamente los Exit Codes.

Cualquier código distinto de 0 debe interpretarse como fallo crítico.

Ingeniero Python:

name: ingeniero_python

isolation_mode: worktree

instruction: Ejecuta scripts de refactorización de datos en árboles aislados.

3.5 Control de Versiones

Gestor Git:

name: gestor_git

isolation_mode: in-process

instruction: Monitorea el estado del repositorio mediante git status.

Capa "So What?": La asignación diferenciada de isolation_mode es clave: usamos in-process para el Gestor Git para maximizar la velocidad de lectura del repositorio, mientras que el Ingeniero Python opera en worktree para evitar que un script experimental altere la rama de producción sin validación previa.

4. Flujo Operativo y Optimización de Tokens en Windows 11

El despliegue sobre el CLI agy (reescrito en Go para máxima eficiencia) requiere un protocolo de despacho riguroso para minimizar el consumo de cuota de la API.

4.1 Protocolo de Despacho Paso a Paso

Para ejecutar cambios globales en las 6 tiendas de pc-custom-lab:

Inicialización: agy init para cargar el árbol de agentes.

Triaje: El Director General analiza la solicitud mediante el pipeline de moldeadores.

Delegación: Se invoca al subagente vía AgentTool.

Ejecución: El subagente opera en su propio worktree o in-process.

Cierre: Retorno de summary-only al orquestador raíz.

Auditoría: El Auditor QA valida los códigos de salida de la operación.

4.2 Estrategias para el Ahorro de Cuota de Conversación

Se prohíbe explícitamente que el orquestador raíz intente realizar tareas monolíticas. La delegación granular mediante AgentTool es obligatoria. Esta estrategia fuerza al sistema a crear ventanas de memoria efímeras que se cierran al completar la subtarea, liberando recursos de contexto.

Capa "So What?": Esta arquitectura de delegación, habilitada por los "Summary-only returns", reduce los costos operativos hasta en un 80%. Al no contaminar la ventana principal con trazas de depuración de subagentes, garantizamos que el orquestador principal retenga un razonamiento preciso y libre de alucinaciones incluso después de horas de actividad continua.

Conclusión: Este manual establece la base para una arquitectura multiagente escalable. La transición a la orquestación declarativa en Antigravity 2.0 no solo mejora la seguridad operativa en Windows 11, sino que permite que pc-custom-lab evolucione mediante la adición de nuevos YAMLS, manteniendo siempre la eficiencia de tokens y la integridad del código en el centro de la operación.